



Person 4: Complete Research and Demo Plan

Your role is:

Environmental data + prior-art research + dashboard/demo development

Martin owns the oil-spill dataset/model.

Jitaan owns the AIS dataset/backtracking.

You support them by collecting environmental sources, references, existing systems, and by building the visual dashboard shell.

Part A: Environmental Data Sources

Environmental data helps answer:

“After the oil was released, where could wind and ocean currents have moved it?”

You mainly need:

- Ocean current direction and speed.
- Wind speed and direction.
- Waves.
- Sea-surface temperature.
- Time and geographic coordinates.

Source 1: Copernicus Marine

Basic information

- **Source:** Copernicus Marine Service.
- **URL:** [Global Ocean Physics Analysis and Forecast](#)
- **Organization:** European Commission / Copernicus Marine Service.
- **Best use:** Ocean-current data for oil-spill drift and backtracking.

Data available

- Eastward ocean-current velocity.
- Northward ocean-current velocity.
- Sea-surface temperature.
- Salinity.
- Sea level.
- Mixed-layer depth.
- Ice-related variables.

The important variables for oil drift are:

```
uo = eastward sea-water velocity  
vo = northward sea-water velocity
```

Resolution

- Approximately $0.083^\circ \times 0.083^\circ$, roughly 8–9 km at the equator.
- Around 50 depth levels.
- Surface current data is the most relevant for floating oil. ^[1]

Time frequency

- Hourly surface fields.
- Daily fields.
- Monthly fields.
- Forecast products are updated daily and provide future ocean conditions. ^[1]

Coverage

- Global ocean.
- Includes:
 - Arabian Sea.
 - Bay of Bengal.
 - Indian coastline.
 - Indian Ocean.

Access method

1. Create an account at Copernicus Marine.
2. Install the Python toolbox:

```
pip install copernicusmarine xarray netcdf4
```

3. Log in:

```
copernicusmarine login
```

4. Download only the required region and time.

Example Python structure

The exact dataset ID and variable name should be confirmed in the Copernicus portal.

```
import copernicusmarine

copernicusmarine.subset(
    dataset_id="VERIFY_DATASET_ID",
    variables=["uo", "vo"],
    minimum_longitude=66.5,
    maximum_longitude=68.5,
    minimum_latitude=19.5,
    maximum_latitude=21.5,
    start_datetime="2026-08-25T00:00:00",
    end_datetime="2026-08-25T12:00:00",
    minimum_depth=0,
    maximum_depth=1,
    output_filename="copernicus_currents.nc"
)
```

Copernicus supports spatial, temporal, and variable subsetting through its Python API. ^[2]

Useful for Jitaan

Jitaan can use uo and vo to move virtual particles:

```
Detected spill location
↓
Read current velocity
↓
Move particles forward or backward
↓
Estimate likely source region
```

Important limitation

- Account required.
- Dataset IDs may change.
- Variable names differ between products.
- Forecast and historical products are not identical.

Mark as:

⚠ Dataset ID and exact variable names must be verified before integration.

Source 2: NOAA ERDDAP

Basic information

- **Source:** NOAA ERDDAP.
- **URL:** [NOAA ERDDAP](#)
- **Organization:** National Oceanic and Atmospheric Administration.
- **Best use:** Wind, waves, SST, and alternative ocean-current data.

Data available

Depending on the selected dataset:

- Wind speed.
- Wind direction.
- Wave height.
- Wave period.
- Sea-surface temperature.
- Surface currents.
- Ocean-colour and marine observations.

NOAA ERDDAP allows users to download subsets in formats such as CSV, JSON, and NetCDF. ^[3]

Resolution

- Depends on the selected NOAA dataset.
- Some products are hourly.
- Others are daily or monthly.
- Some current products provide daily gridded surface velocities. ^[4]

Coverage

- Global or regional, depending on product.
- Can cover:
 - Arabian Sea.
 - Bay of Bengal.
 - Indian Ocean.
 - Indian coastal waters.

Access method

Use the ERDDAP web interface or API.

Typical ERDDAP structure:

```
https://server/erddap/griddap/DATASET_ID.nc?
```

A specific query normally selects:

```
variable[time][latitude][longitude]
```

Example concept

```
Download:  
wind_speed[date][lat range][lon range]  
wind_direction[date][lat range][lon range]  
wave_height[date][lat range][lon range]
```

Useful for Jitaan

- Add wind effects to current-based drift.
- Validate Copernicus current direction.
- Display environmental conditions in the dashboard.
- Show why the predicted oil movement follows a particular direction.

Important limitation

NOAA has many datasets with different names, resolutions, and access rules.

Mark as:

⚠ Select and test one specific ERDDAP dataset before the final demo.

Source 3: INCOIS

Basic information

- **Source:** Indian National Centre for Ocean Information Services.
- **URL:** [INCOIS Data Holdings](#)
- **Organization:** Ministry of Earth Sciences, Government of India.
- **Best use:** Indian Ocean and Indian coastal validation.

Data available

- Sea-surface temperature.
- Ocean currents.
- Wind parameters.
- Waves.
- Buoy observations.
- Current-meter observations.
- HF radar currents.
- Coastal observations.

INCOIS lists drifting buoy data containing SST and ocean currents, moored-buoy data containing wind, currents, SST and wave parameters, and other Indian Ocean observations. ^[5]

Resolution

- Depends on the platform:
 - Buoy data are point observations.
 - HF radar provides coastal current vectors.
 - Model and satellite products have product-specific grids.

Time period

Examples listed by INCOIS include:

- Drifting buoy data: 1991 to present.
- Moored buoy data: 1997 to present.
- HF radar current data: 2008 to present.
- INCOIS AVHRR SST: 2004 to present. ^[5]

Coverage

- Indian Ocean.
- Arabian Sea.
- Bay of Bengal.
- Indian coastal regions.

Format

Depending on the product:

- ASCII.
- NetCDF.
- HDF.
- Binary formats.

Access method

Some INCOIS data provide:

- Visualization.
- Download options.
- Registered access.

Other products provide visualization only. For example, INCOIS states that drifting-buoy data have visualization and download options, while some moored-buoy and wave-rider datasets have no direct download option. ^[5]

Useful for Jitaan

- Indian-region environmental validation.
- Checking whether Copernicus currents are reasonable.
- Adding local observations to strengthen the presentation.
- Demonstrating India-specific relevance.

Important limitation

⚠ Access depends on the specific INCOIS product. Do not promise direct API access without testing it.

Environmental Data Recommendation

For the hackathon, use:

Need	Primary source	Backup
Ocean currents	Copernicus Marine	NOAA ERDDAP
Wind	NOAA ERDDAP	Copernicus Marine wind product
Waves	NOAA ERDDAP	INCOIS
Indian coastal validation	INCOIS	Copernicus Marine
SST	INCOIS or Copernicus	NOAA

Part B: GitHub Repositories

1. Oil-Spill-Detection

- **Repository:** chashmishcoder/Oil-Spill-Detection
- **URL:** <https://github.com/chashmishcoder/Oil-Spill-Detection>
- **Purpose:** Oil-spill detection using Sentinel-1 SAR and AIS.
- **Language:** Python.
- **Approach:** U-Net segmentation and DBSCAN-based AIS anomaly analysis.
- **Reusable components:**
 - AIS feature extraction.
 - U-Net segmentation workflow.
 - Anomaly clustering.
 - Combining satellite and vessel evidence.

The repository describes a workflow using AIS features such as speed over ground and course over ground, followed by U-Net segmentation of Sentinel-1 imagery. ^[6]

⚠ Verify license and dependencies before copying code.

2. Oil-Spill-Detection

- **Repository:** Misash/Oill-Spill-Detection
- **URL:** <https://github.com/Misash/Oill-Spill-Detection>
- **Purpose:** U-Net-based oil-spill detection.
- **Language:** Python.
- **Framework:** TensorFlow/Keras likely; verify files.
- **Reusable components:**

- Training notebooks.
- U-Net architecture.
- Image preprocessing.
- Segmentation-mask visualization.

The project was created for detecting marine oil in satellite imagery related to the Ventanilla, Peru spill. ^[7]

⚠ License and current compatibility need verification.

3. RS-xjg Oil-Spill Detection

- **Repository:** RS-xjg/oil-spill-detection
- **URL:** <https://github.com/RS-xjg/oil-spill-detection>
- **Purpose:** Oil-spill detection from Sentinel-1 SAR.
- **Language:** Python.
- **Approach:** Deep convolutional model using SAR scattering information.
- **Reusable components:**
 - SAR input processing.
 - Polarization/scattering feature ideas.
 - Deep-learning detection pipeline.

^[8]

4. OceanHackWeek Oil Project

- **Repository:** oceanhackweek/ohw23_proj_oil
- **URL:** https://github.com/oceanhackweek/ohw23_proj_oil
- **Purpose:** Detect oil slicks and distinguish them from look-alikes.
- **Language:** Python notebooks.
- **Approach:** Crop SAR images, use image statistics, and inspect likely slick patches.
- **Reusable components:**
 - SAR image cropping.
 - Histogram analysis.
 - Look-alike inspection.
 - Quick exploratory visualizations.

This is particularly useful for building a fast baseline before the deep-learning model is connected. ^[9]

5. Maritime Vessel Tracking

- **Repository:** `josna-14/Maritime_Vessel_Tracking`
- **URL:** https://github.com/josna-14/Maritime_Vessel_Tracking
- **Purpose:** AIS vessel tracking, port-congestion analysis, and maritime safety alerts.
- **Language:** JavaScript and Python.
- **Framework:** React, Django, Leaflet, Axios, WebSockets.
- **Reusable components:**
 - Vessel map.
 - Sidebar layout.
 - Marker clustering.
 - Vessel route replay.
 - API and WebSocket structure.

The repository describes a React-Leaflet frontend with vessel clustering and real-time AIS updates. ^[10]

6. Vessel Tracks

- **Repository:** `kboul/vessel-tracks`
- **URL:** <https://github.com/kboul/vessel-tracks>
- **Purpose:** Display and animate vessel tracks.
- **Language:** JavaScript.
- **Framework:** React and React-Leaflet.
- **Reusable components:**
 - Animated tracks.
 - Polyline display.
 - Vessel movement visualization.
 - Map controls.

^[11]

7. React-Leaflet

- **Repository:** `PaulLeCam/react-leaflet`
- **URL:** <https://github.com/PaulLeCam/react-leaflet>
- **Purpose:** React components for Leaflet maps.
- **Language:** JavaScript/TypeScript.
- **Reusable components:**

- Map container.
- Markers.
- Polylines.
- Polygons.
- Popups.
- GeoJSON layers.

React-Leaflet is useful if your team wants a proper React frontend instead of a plain HTML/JavaScript prototype. ^[12]

8. VaruNetra

- **Repository:** GlenElric/VaruNetra
- **URL:** <https://github.com/GlenElric/VaruNetra>
- **Purpose:** Real-time maritime intelligence and naval situational awareness.
- **Language:** JavaScript.
- **Framework:** React, Vite, Leaflet.
- **Data:** [AISStream.io](https://aisstream.io) WebSocket.
- **Reusable components:**
 - Tactical map design.
 - Vessel markers.
 - Marker clustering.
 - Collapsible information panels.
 - Real-time vessel updates.

^[13]

Part C: Papers and Prior Art

Paper/System 1: CleanSeaNet

- **Title:** CleanSeaNet — European Satellite-Based Oil Spill Monitoring and Vessel Detection Service.
- **URL:** [EMSA CleanSeaNet](https://www.emsa.europa.eu/cleanseanet)
- **Organization:** European Maritime Safety Agency.
- **What it does:** Detects possible oil pollution using SAR and optical satellite images.
- **Other data:** AIS, vessel detections, meteorological information, and oceanographic information.
- **Useful to us:** It proves that satellite detection and vessel attribution are already used in real maritime operations.

CleanSeaNet provides spill location, spill area, detection confidence, and information about possible vessel or oil-and-gas-installation sources. ^[14]

Gap

- Primarily supports European maritime authorities.
- It is not an open India-focused student platform.
- Its complete operational pipeline is not publicly available.

Our difference

Open and low-cost
India-focused
Explainable ranking
Dashboard for judges

Paper/System 2: Automatic SAR Oil-Spill Detection and CleanSeaNet

- **Title:** Automatic Synthetic Aperture Radar-Based Oil Spill Detection and Its Application to CleanSeaNet.
- **URL:** [ScienceDirect](#)
- **What it does:** Uses SAR images and supporting meteorological, oceanographic, and AIS data.
- **Useful to us:** Strong prior art for combining image evidence, environmental data, and ship information.

The study explains that satellite images can be analyzed with meteorological, oceanographic, and AIS data to estimate whether a dark feature is oil and help identify its source. ^[15]

Paper/System 3: Sentinel-1 Oil-Spill Mapping

- **Title:** Oil Spill Mapping Using Sentinel-1.
- **URL:** [Copernicus Documentation](#)
- **What it does:** Demonstrates processing Sentinel-1 imagery to map possible oil spills.
- **Useful to us:** Helps Martin create or explain the satellite-image processing stage.

^[16]

Paper 4: Dual-Polarimetric Sentinel-1 Oil-Spill Detection

- **Title:** Oil Spill Detection with Dual-Polarimetric Sentinel-1 SAR Using Superpixel-Level Image Stretching and Deep Convolutional Neural Network.
- **URL:** [Remote Sensing](#)
- **Year:** 2022.
- **What it does:** Uses Sentinel-1 VV/VH information, image enhancement, superpixels, and CNN.
- **Useful to us:** Supports using VV/VH channels and preprocessing before segmentation.

The reported experiments found that image stretching improved mean IoU compared with the version without stretching, although results were dataset-specific. ^[17]

Paper 5: Deep-Learning Framework for Oil-Spill Detection

- **Title:** A Deep-Learning Framework for the Detection of Oil Spills from SAR Data.
- **URL:** [Sensors](#)
- **Year:** 2021.
- **What it does:** Uses a two-stage system:
 1. Classifies image patches.
 2. Segments the oil using U-Net.
- **Dataset:** Sentinel-1 imagery.
- **Useful to us:** Supports separating “is there oil?” from “where exactly is the oil?”

The paper uses generalized Dice loss because oil pixels are much less common than background pixels. ^[18]

Paper 6: GSCAT-UNET

- **Title:** GSCAT-UNET: Enhanced U-Net Model with Spatial-Channel Attention Gate and Three-Level Attention for Oil Spill Detection Using SAR Data.
- **URL:** [PubMed](#)
- **Year:** 2025.
- **What it does:** Adds spatial and channel attention to U-Net.
- **Dataset:** Sentinel-1 dual-polarization data with 1,112 images and five labelled classes.
- **Useful to us:** Provides a more advanced U-Net idea for distinguishing oil, look-alikes, ships, land, and sea.

Paper 7: New Ship-Tracing Technology from Oil Spills

- **Title:** A New Ship Tracing Technology from Oil Spills Based on Multi-Source Data.
- **URL:** [PubMed](#)
- **Year:** 2024.
- **What it does:** Uses satellite data, AIS, and marine-environment data to trace a spill to a probable source vessel.
- **Useful to us:** This is one of the closest references to your team's proposed architecture.

Use it to justify:

```
Satellite detection
+
AIS route matching
+
Environmental drift
=
Probable source attribution
```

Paper 8: OilSpillNets

- **Title:** Oilspillnets: Detecting and Estimating Oil Spills Using Multi-Source Imagery.
- **URL:** [Springer DOI](#)
- **Year:** 2025.
- **What it does:** Uses satellite SAR and high-resolution drone imagery for oil-spill detection and estimation.
- **Useful to us:** Supports multi-source image fusion and future expansion of the project.

[20]

Part D: Existing Systems and Their Gaps

CleanSeaNet

What it does

- Detects potential oil spills.
- Uses SAR and optical satellite images.
- Detects vessels.
- Uses AIS and environmental information.
- Supports pollution response.
- Helps identify possible polluters.

What it does not give your team

- Open, simple, student-friendly implementation.
- India-focused public dashboard.
- Transparent source-ranking logic for hackathon users.

[\[14\]](#) [\[21\]](#)

Cerulean

Organization

SkyTruth, with Global Fishing Watch support.

What it does

- Scans Sentinel-1 imagery.
- Uses a U-Net model.
- Detects oil slicks.
- Uses AIS vessel histories.
- Calculates proximity, timing, and path alignment.
- Connects slicks to vessels or offshore infrastructure.

Cerulean currently describes a ResNet-34 U-Net model using Sentinel-1 VV imagery. It evaluates vessel candidates using slick-path alignment, proximity, and timing. [\[22\]](#)

Gap

Your project should not claim to replace Cerulean. Instead, differentiate through:

- India-focused use case.
- Local INCOIS environmental information.
- Student-friendly open architecture.
- Explainable backtracking visualization.
- Integration with your team's own AIS and model outputs.

INCOIS

What it does

- Provides Indian Ocean environmental information.
- Provides currents, SST, waves, wind, and buoy data.
- Supports ocean forecasting and marine services.

Gap

INCOIS is not primarily an end-to-end oil-spill detection and vessel-attribution dashboard.

Your opportunity

```
INCOIS environment data
+
Sentinel-1 oil mask
+
AIS vessel tracks
+
Backtracking
+
Explainable dashboard
```

Part E: Dashboard Design

Recommended Technology

Use:

```
Frontend: React or plain HTML/JavaScript
Map: Leaflet
Map data: GeoJSON
Charts: Recharts or simple HTML
Backend: Optional FastAPI
Data exchange: JSON
```

Leaflet supports markers, polygons, polylines, popups, and GeoJSON layers, making it sufficient for the SIH prototype. [\[23\]](#) [\[24\]](#)

Why Choose Leaflet?

- Free and open-source.
- No Mapbox token required.
- Easy to learn.
- Works with React.
- Supports GeoJSON.
- Good for spill polygons and vessel routes.

- Good for a quick demo.

Use proper OpenStreetMap attribution and avoid bulk tile downloading or offline caching. ^[25]

Dashboard Layout

+-----+	
SIH26143 Oil-Spill Detection and Attribution	
+-----+	
MAP	
Spill polygon + source area + vessel tracks	
+-----+	
	Incident Summary
	Spill confidence
	Estimated area
	Environmental data
	Ranked vessels
	Attribution reason
+-----+	

Map Layers

Layer 1: Base map

OpenStreetMap tiles.

Layer 2: Oil-spill polygon

- Red polygon.
- Popup with spill area and detection confidence.

Layer 3: Backtracked source region

- Orange circle or dashed polygon.
- Popup with estimated source time window.

Layer 4: Vessel tracks

- Blue lines for normal candidates.
- Red line for the highest-ranked candidate.

Layer 5: Vessel markers

- Blue markers for normal ships.
- Red marker for the flagged candidate.
- Different icons can represent cargo ships, tankers, and fishing vessels.

Layer 6: Environmental arrows

Optional:

- Blue arrows for ocean-current direction.
- Green arrows for wind.
- Popup showing speed in m/s.

Suggested API Data Format

Tell the other team members to return data in this structure:

```
{
  "incident": {
    "id": "SPILL-001",
    "detected_at": "2026-08-25T06:00:00Z",
    "area_km2": 3.2,
    "confidence": 0.87,
    "polygon": [
      [67.15, 20.45],
      [67.45, 20.75],
      [67.90, 20.62],
      [67.70, 20.30],
      [67.35, 20.25]
    ]
  },
  "environment": {
    "current_u_ms": 0.18,
    "current_v_ms": 0.07,
    "wind_speed_ms": 5.4,
    "wind_direction_deg": 72,
    "source_model": "Copernicus Marine"
  },
  "source_region": {
    "latitude": 20.48,
    "longitude": 67.52,
    "radius_km": 22,
    "backtrack_hours": 24
  },
  "vessels": [
    {
      "name": "MV Ocean Star",
      "mmsi": "419001234",
      "type": "Tanker",
      "confidence": 0.86,
```

```

    "distance_km": 8.4,
    "time_difference_hours": 3,
    "reason": "Passed near the backtracked source region",
    "position": [67.10, 20.15],
    "track": [
      [67.10, 20.15],
      [66.90, 20.00],
      [66.70, 19.85]
    ]
  }
]
}

```

Important GeoJSON rule:

```
[longitude, latitude]
```

is the standard coordinate order.

Minimal Demo Code

File structure

```

sih26143-dashboard/
├─ index.html
├─ style.css
└─ app.js

```

index.html

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SIH26143 Oil-Spill Dashboard</title>

  <link
    rel="stylesheet"
    href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"
  >

  <link rel="stylesheet" href="style.css">
</head>

<body>
  <header>
    <h1>SIH26143 Oil-Spill Monitoring</h1>
    <p>Environmental Data + AIS Vessel Attribution</p>
  </header>

```

```

<main>
  <section id="map"></section>

  <aside>
    <h2>Incident Summary</h2>

    <div class="card">
      <b>Status:</b> Suspected oil spill<br>
      <b>Area:</b> 3.2 km²<br>
      <b>Detection confidence:</b> 87%
    </div>

    <h2>Environmental Conditions</h2>

    <div class="card">
      <b>Current:</b> 0.19 m/s<br>
      <b>Wind:</b> 5.4 m/s<br>
      <b>Source:</b> Copernicus Marine
    </div>

    <h2>Ranked Candidate Vessels</h2>
    <div id="candidates"></div>

    <p class="warning">
      Attribution is probabilistic and does not prove legal responsibility.
    </p>
  </aside>
</main>

<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="app.js"></script>
</body>
</html>

```

style.css

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #172b4d;
}

header {
  padding: 14px 20px;
  background: #0b2d4d;
  color: white;
}

header h1 {
  margin: 0;
  font-size: 22px;
}

header p {

```

```
    margin: 5px 0 0;
}

main {
  display: grid;
  grid-template-columns: 1fr 350px;
  height: calc(100vh - 80px);
}

#map {
  height: 100%;
}

aside {
  padding: 16px;
  overflow-y: auto;
  background: white;
  box-shadow: -2px 0 8px #0002;
}

.card,
.candidate {
  padding: 12px;
  margin: 10px 0;
  border: 1px solid #dce3ec;
  border-radius: 8px;
}

.candidate {
  border-left: 5px solid #2563eb;
  background: #f4f7fb;
}

.flagged {
  border-left-color: #d92d20;
}

.confidence {
  display: inline-block;
  margin-top: 6px;
  padding: 4px 8px;
  border-radius: 12px;
  background: #e6f4ea;
  color: #087443;
}

.warning {
  color: #9b2c2c;
  font-size: 13px;
}

@media (max-width: 800px) {
  main {
    display: block;
    height: auto;
  }
}
```

```

    #map {
      height: 60vh;
    }

    aside {
      height: auto;
    }
  }
}

```

app.js

```

const map = L.map("map").setView([20.5, 67.5], 6);

L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
  attribution: "© OpenStreetMap contributors"
}).addTo(map);

const spill = [
  [20.45, 67.15],
  [20.75, 67.45],
  [20.62, 67.90],
  [20.30, 67.70],
  [20.25, 67.35]
];

L.polygon(spill, {
  color: "#d92d20",
  fillColor: "#f04438",
  fillOpacity: 0.45,
  weight: 3
})
.addTo(map)
.bindPopup(`
  <b>Suspected oil spill</b><br>
  Area: 3.2 km2<br>
  Confidence: 87%
`);

L.circle([20.48, 67.52], {
  radius: 22000,
  color: "#f79009",
  fill: false,
  dashArray: "8 8"
})
.addTo(map)
.bindTooltip("Probable backtracked source region");

const vessels = [
  {
    name: "MV Ocean Star",
    mmsi: "419001234",
    type: "Tanker",
    score: 86,
    position: [20.15, 67.10],
  }
]

```

```

        track: [
            [19.70, 66.40],
            [19.85, 66.70],
            [20.00, 66.90],
            [20.15, 67.10]
        ],
        reason: "Passed near the backtracked source region."
    },
    {
        name: "MT Gujarat Pearl",
        mmsi: "419005678",
        type: "Cargo",
        score: 64,
        position: [21.10, 67.70],
        track: [
            [21.40, 67.20],
            [21.30, 67.35],
            [21.20, 67.55],
            [21.10, 67.70]
        ],
        reason: "Nearby route, but weaker timing evidence."
    }
];

const candidateBox = document.getElementById("candidates");

vessels.forEach((vessel, index) => {
    const flagged = index === 0;
    const color = flagged ? "#d92d20" : "#2563eb";

    L.polyline(vessel.track, {
        color: color,
        weight: flagged ? 5 : 3
    })
    .addTo(map)
    .bindPopup(`
        <b>${vessel.name}</b><br>
        Type: ${vessel.type}<br>
        MMSI: ${vessel.mmsi}<br>
        Candidate score: ${vessel.score}%
    `);

    L.circleMarker(vessel.position, {
        radius: flagged ? 9 : 6,
        color: color,
        fillColor: color,
        fillOpacity: 1
    })
    .addTo(map)
    .bindTooltip(vessel.name);

    candidateBox.innerHTML += `
        <div class="candidate ${flagged ? "flagged" : ""}">
            <b>${flagged ? "☒ " : ""}${vessel.name}</b><br>
            Type: ${vessel.type}<br>
            MMSI: ${vessel.mmsi}<br>

```

```

        <span class="confidence">
          Candidate score: ${vessel.score}%
        </span>
        <p>${vessel.reason}</p>
      </div>
    `;
  });

  map.fitBounds(L.polygon(spill).getBounds().pad(0.8));

```

Leaflet officially supports markers, polygons, polylines, popups, and GeoJSON layers for this type of map interface. [\[23\]](#) [\[24\]](#)

How to Run the Demo

Open a terminal inside the project folder:

```
python -m http.server 8000
```

Open:

```
http://localhost:8000
```

If Python is not installed, use VS Code Live Server.

Demo-to-Real-Data Integration

Tomorrow, replace the dummy objects in `app.js` with a request to your backend:

```

fetch("http://localhost:8000/api/incident/SPILL-001")
  .then(response => response.json())
  .then(data => {
    drawSpill(data.incident);
    drawSourceRegion(data.source_region);
    drawVessels(data.vessels);
    updateEnvironment(data.environment);
  });

```

Your backend can be built with FastAPI:

```

GET /api/incident/SPILL-001
GET /api/vessels?bbox=...
GET /api/environment?lat=...&lon=...&time=...

```


What Each Teammate Must Provide

Martin

Provide:

```
spill polygon
spill area
segmentation mask
model confidence
detection timestamp
```

Jitaan

Provide:

```
vessel name
MMSI
vessel type
AIS position
historical track
distance from source region
time difference
candidate score
```

Environmental module

Provide:

```
current_u
current_v
wind_speed
wind_direction
data timestamp
source name
backtracked source region
```

Person 4 dashboard

Display all of this in:

```
Map
+
Sidebar
+
Popups
+
Candidate ranking
```

Two-Day Action Plan

Tonight

1. Create the Leaflet dashboard.
2. Add the dummy spill polygon.
3. Add two or three dummy vessel tracks.
4. Highlight one flagged vessel.
5. Add confidence badges.
6. Add the environmental-data card.
7. Create and test the JSON format.

Tomorrow Morning

1. Connect Martin's spill polygon.
2. Connect Jitaan's vessel-track JSON.
3. Add the real backtracked source region.
4. Add current and wind values.
5. Add candidate-vessel scores.
6. Test the dashboard on a laptop and projector.
7. Take screenshots for the presentation.

Tomorrow Evening

1. Improve colors and labels.
2. Add a legend.
3. Add a "Detection Details" popup.
4. Add a "Why this vessel?" explanation box.
5. Prepare a 60-second demo script.

One-Minute Demo Script

“This dashboard shows a suspected oil spill detected from Sentinel-1 satellite imagery. The red polygon represents the detected spill, while the orange circle represents the area estimated by backtracking ocean currents and wind. Blue lines show AIS vessel routes. The system compares vessel location, timing, route direction, and distance from the estimated source region. The red vessel is the highest-ranked candidate, but the score represents probable attribution—not legal proof. Environmental data comes from Copernicus Marine, while the system is designed for Indian waters such as the Arabian Sea and Bay of Bengal.”

Important Novelty Statement

Existing systems such as CleanSeaNet and Cerulean already demonstrate satellite-based oil-spill detection with AIS-based source analysis. ^[14] ^[22]

Your novelty should not be:

“We are the first system to combine satellites and AIS.”

Instead, say:

“We propose a low-cost, India-focused, explainable dashboard that combines Sentinel-1 oil-spill segmentation, AIS vessel tracks, Indian Ocean environmental data, and visual drift backtracking to rank probable source vessels.”

That claim is safer, more realistic, and better suited to your SIH presentation.

✱✱

1. https://data.marine.copernicus.eu/product/GLOBAL_ANALYSISFORECAST_PHY_001_024/description
2. <https://help.marine.copernicus.eu/en/articles/8283072-copernicus-marine-toolbox-api-subset>
3. <https://www.ncei.noaa.gov/erddap/index.html>
4. <https://coastwatch.noaa.gov/erddap/info/noaacwBLENDEDNRTcurrentsDaily/index.html>
5. <https://incois.gov.in/site/dataholdings.jsp>
6. <https://github.com/chashmishcoder/Oil-Spill-Detection>
7. <https://github.com/Misash/Oil-Spill-Detection>
8. <https://github.com/RS-xjg/oil-spill-detection>
9. https://github.com/oceanhackweek/ohw23_proj_oil
10. https://github.com/josna-14/Maritime_Vessel_Tracking
11. <https://github.com/kboul/vessel-tracks>
12. <https://github.com/paullecam/react-leaflet>
13. <https://github.com/GlenElric/VaruNetra>
14. <https://www.emsa.europa.eu/csn-menu.html>
15. <https://www.sciencedirect.com/science/article/abs/pii/S0025326X13002695>

16. <https://documentation.dataspace.copernicus.eu/APIs/openEO/openeo-community-examples/python/OilSpill/OilSpillMapping.html>
17. <https://www.mdpi.com/2072-4292/14/16/3900>
18. <https://www.mdpi.com/1424-8220/21/7/2351>
19. <https://pubmed.ncbi.nlm.nih.gov/39862681/>
20. https://link.springer.com/article/10.1007/s41207-025-00914-y?error=cookies_not_supported&code=84f90418-8f46-460f-9ea4-0c7875a36d4d
21. <https://www.emsa.europa.eu/publications/item/4322-cleanseanet-european-satellite-based-oil-spill-monitoring-and-vessel-detection-service.html>
22. <https://skytruth.org/cerulean/methods>
23. <https://leafletjs.com/examples/geojson/>
24. <https://leafletjs.com/reference.html>
25. <https://scripts.nuxt.com/scripts/leaflet/guides/tiles-and-attribution.md>
26. <https://incois.gov.in/portal/datainfo/dataholdings.jsp>
27. <https://marine.copernicus.eu/access-data/>
28. <https://incois.gov.in/site/services/rsmc.jsp>
29. https://data.marine.copernicus.eu/product/GLOBAL_MULTIYEAR_PHY_001_030/services
30. https://data.marine.copernicus.eu/product/GLOBAL_ANALYSISFORECAST_PHY_001_024/services
31. https://data.marine.copernicus.eu/product/GLOBAL_MULTIYEAR_PHY_001_030/description
32. https://data.marine.copernicus.eu/product/INSITU_GLO_PHY_UV_DISCRETE_NRT_013_048/description
33. <https://github.com/topics/oil-spills>
34. <https://github.com/satellite-image-deep-learning/techniques>
35. <https://github.com/juncyan/landslide>
36. <https://github.com/linh22/deep-learning-techniques>
37. <https://github.com/satellite-image-deep-learning/datasets>
38. <https://publications.lib.chalmers.se/records/fulltext/141265.pdf>
39. <https://www.atlantis-press.com/proceedings/eamcon-25/126020768>
40. <https://www.proceedings.blucher.com.br/article-details/oil-spill-detection-utilizing-unet-r-image-segmentation-and-sentinel-i-data-38924>
41. <https://www.atlantis-press.com/article/126020768.pdf>
42. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12914053/>
43. <https://meetingorganizer.copernicus.org/EGU2011/EGU2011-13349.pdf>
44. <https://pmc.ncbi.nlm.nih.gov/articles/PMC13324850/>
45. <https://geodata.tools/blog/leaflet-geojson-layer-tutorial>
46. <https://leafletjs.com/examples.html>
47. <https://leafletjs.com/reference-2.0.0.html>
48. <https://leafletjs.com/examples/quick-start/>

49. <https://community.openstreetmap.org/t/websites-using-osmf-tile-servers-without-attribution-can-now-be-easily-reported/107952>
50. <https://github.com/iamtekson/Leaflet-from-basic-to-advance>
51. https://www.dash-leaflet.com/docs/geojson_tutorial
52. <https://skytruth.org/cerulean>
53. <https://skytruth.org/2025/01/leveling-up-ceruleans-ability-to-reveal-stationary-polluters/>
54. <https://skytruth.org/blog/near-real-time-coming-to-cerulean>
55. <https://www.emsa.europa.eu/publications/leaflets/download/2913/2123/23.html>
56. <https://skytruth.org/product-updates/dark-vessels-as-sources>
57. <https://www.emsa.europa.eu/csn-menu/items.html?cid=122&id=4322>
58. <https://pubmed.ncbi.nlm.nih.gov/39146713/>